

Menu-Driven Matrix Distributive Properties

C Programming Lab - Variable Length 2D Arrays and Separate Functions

Problem Statement

Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D arrays and verify the left distributive property: $A * (B + C) = (A * B) + (A * C)$.
Dimensions: A (m x n), B (n x p), C (n x p).
2. Read three variable-length 2D arrays and verify the right distributive property: $(A + B) * C = (A * C) + (B * C)$.
Dimensions: A (m x n), B (m x n), C (n x p).
3. Exit.

Objective

To implement matrix addition and multiplication using separate functions and verify the left and right distributive properties of matrix multiplication through a menu-driven C program using variable-length arrays (VLAs).

Algorithm

1. Display a menu for left distributive property, right distributive property, and exit.
2. For left distributive property, read m, n and p; then input A[m][n], B[n][p] and C[n][p].
3. Compute B+C, then A*(B+C). Separately compute A*B and A*C, then add the two products.
4. Compare the two resulting m x p matrices element by element and display whether the property is verified.
5. For right distributive property, read m, n and p; then input A[m][n], B[m][n] and C[n][p].
6. Compute A+B, then (A+B)*C. Separately compute A*C and B*C, then add the two products.
7. Compare the two resulting m x p matrices and display whether the property is verified.
8. Repeat until the user selects Exit.

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c], char name) {
    printf("Enter elements of matrix %c (%d x %d):\n", name, r, c);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void addMatrix(int r, int c, int a[r][c], int b[r][c], int sum[r][c]) {
    for (int i = 0; i < r; i++)
```

```

        for (int j = 0; j < c; j++)
            sum[i][j] = a[i][j] + b[i][j];
    }

void multiplyMatrix(int r1, int c1, int c2,
                    int a[r1][c1], int b[c1][c2], int product[r1][c2]) {
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            product[i][j] = 0;
            for (int k = 0; k < c1; k++)
                product[i][j] += a[i][k] * b[k][j];
        }
    }
}

int equalMatrix(int r, int c, int a[r][c], int b[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

void printMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) printf("%d\t", a[i][j]);
        printf("\n");
    }
}

void verifyLeft(void) {
    int m, n, p;
    printf("Enter m n p: ");
    scanf("%d%d%d", &m, &n, &p);
    int A[m][n], B[n][p], C[n][p];
    int BC[n][p], left[m][p], AB[m][p], AC[m][p], right[m][p];

    readMatrix(m, n, A, 'A');
    readMatrix(n, p, B, 'B');
    readMatrix(n, p, C, 'C');
    addMatrix(n, p, B, C, BC);
    multiplyMatrix(m, n, p, A, BC, left);
    multiplyMatrix(m, n, p, A, B, AB);
    multiplyMatrix(m, n, p, A, C, AC);
    addMatrix(m, p, AB, AC, right);

    printf("\nA*(B+C):\n"); printMatrix(m, p, left);
    printf("\n(A*B)+(A*C):\n"); printMatrix(m, p, right);
    printf(equalMatrix(m, p, left, right) ?
           "\nLeft distributive property is VERIFIED.\n" :
           "\nLeft distributive property is NOT verified.\n");
}

void verifyRight(void) {
    int m, n, p;
    printf("Enter m n p: ");
    scanf("%d%d%d", &m, &n, &p);

```

```

int A[m][n], B[m][n], C[n][p];
int AB[m][n], left[m][p], AC[m][p], BC[m][p], right[m][p];

readMatrix(m, n, A, 'A');
readMatrix(m, n, B, 'B');
readMatrix(n, p, C, 'C');
addMatrix(m, n, A, B, AB);
multiplyMatrix(m, n, p, AB, C, left);
multiplyMatrix(m, n, p, A, C, AC);
multiplyMatrix(m, n, p, B, C, BC);
addMatrix(m, p, AC, BC, right);

printf("\n(A+B)*C:\n"); printMatrix(m, p, left);
printf("\n(A*C)+(B*C):\n"); printMatrix(m, p, right);
printf(equalMatrix(m, p, left, right) ?
        "\nRight distributive property is VERIFIED.\n" :
        "\nRight distributive property is NOT verified.\n");
}

int main(void) {
    int choice;
    do {
        printf("\n--- MENU ---\n");
        printf("1. Verify left distributive property\n");
        printf("2. Verify right distributive property\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: verifyLeft(); break;
            case 2: verifyRight(); break;
            case 3: printf("Exiting program.\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 3);
    return 0;
}

```

Suggested Test Data

Use $m = 2$, $n = 2$, $p = 2$ for simple screenshots.

Matrix	Row 1	Row 2	Use in
A	1 2	3 4	Both options
B	5 6	7 8	Both options
C	1 0	0 1	Both options

Expected Result

For valid conformable matrices, both sides of each distributive identity should produce identical result matrices. The program should print VERIFIED for the selected property.

Execution Screenshots

Compilation - Successful compilation without errors

```
(kali㉿kali)-[~/Desktop/apple]
$ gcc hdoc34.c -o hdoc34

(kali㉿kali)-[~/Desktop/apple]
$
```

Output 1 - Left Distributive Property Verified

```
— MENU —
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 1
Enter m n p: 2 2 2
Enter elements of matrix A (2 x 2):
1 2 3 4
Enter elements of matrix B (2 x 2):
5 6 7 8
Enter elements of matrix C (2 x 2):
1 2 3 4

A*(B+C):
26      32
58      72

(A*B)+(A*C):
26      32
58      72

Left distributive property is VERIFIED.
```

Output 2 - Right Distributive Property Verified

```
— MENU —
1. Verify left distributive property
2. Verify right distributive property
3. Exit
Enter your choice: 2
Enter m n p: 2 2 2
Enter elements of matrix A (2 x 2):
1 2 3 4
Enter elements of matrix B (2 x 2):
5 6 7 8
Enter elements of matrix C (2 x 2):
1 2 3 4

(A+B)*C:
30      44
46      68

(A*C)+(B*C):
30      44
46      68

Right distributive property is VERIFIED.
```